Optimisation du temps de réaction des sportifs grâce à l'entraînement visuel

Comment tester le temps de réaction d'un individu ?

Plan de la présentation

01-Présentation du problème

Contextualisation, enjeux et objectifs

02-Conception

Cahier des charges, choix des composants et conception d'un prototype

03-Expérimentations

Tests du prototype, fabrication et tests du produit final, comparaison des résultats

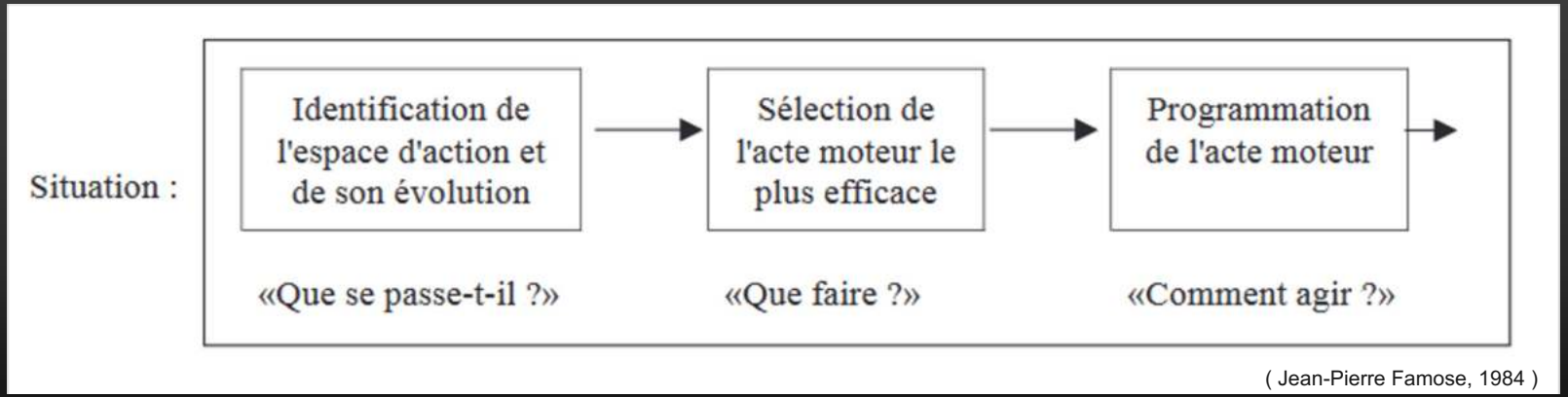
04-Bilan

Observations, conclusions et réponse à la problématique

Le temps de réaction, un réflexe ?

- Percevoir, traiter, agir
- Temps de réaction simple, de reconnaissance ou cognitif
- La clé de la réussite sportive
- Tennis, Football, Basketball, Automobile…

Modèles cognitifs du traitement de l'information



(Jean-Pierre Famose, 1984)

Modèle de Welford sur l'approche cognitive, proposé en 1968

Vision	Interprétation	Mouvement
Perception des couleurs, de la profondeur et des distances Sensibilité au contraste Acuité visuelle dynamique Accommodation Eye tracking Suivi de plusieurs objets Vision périphérique	Vitesse de compréhension et de traitement des stimulis Anticipation	Coordination oeil-main Temps de réaction

170ms (2023, Pierre Gasly)

- 2 à 3 fois moins élevé que pour un individu normal (sécurité routière, 2015)
- Des centaines d'accidents évités grâce à un temps de réaction faible
- Importance de travailler le temps de réaction des sportifs

Conception

1) Mettre en place un cahier des charges
2) Choisir les composants adaptés respectant le cahier des charges
3) Concevoir le circuit électrique
4) Concevoir le boîtier

Cahier des charges

1) Permettre une amélioration du temps de réaction de l'utilisateur
2) Permettre un relevé temporel du temps de réponse précis
3) Communiquer les relevés temporels pour un éventuel stockage et des analyses
4) Avoir une autonomie d'au moins un à deux jours
5) Être facilement transportable
6) Être abordable (60€ maximum)

Choix des composants



Processeur Arduino

Accessible, précis à 4µs près, puissance de calcul élevée



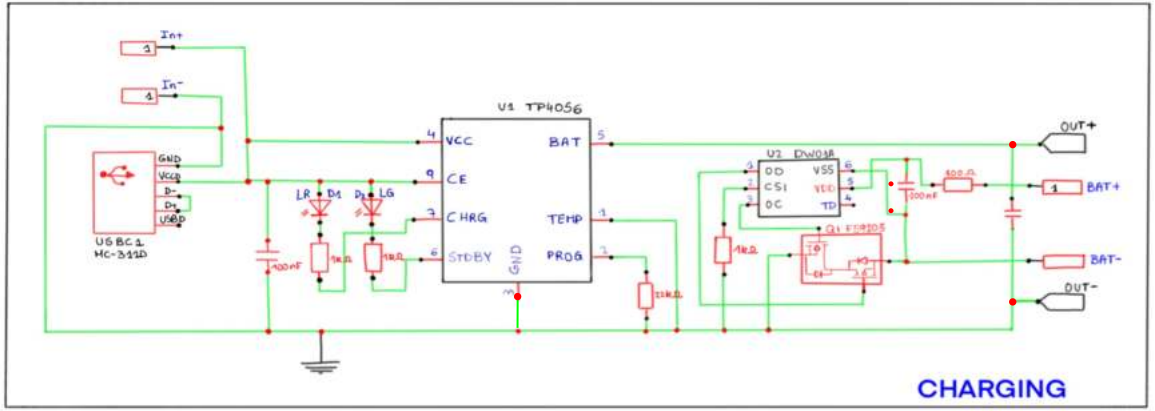
Module GSM, SIM800C

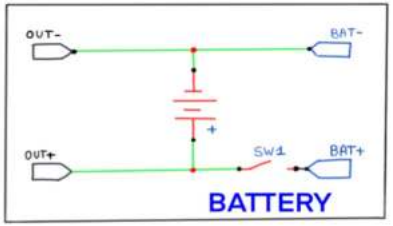
Grande portée, faible consommation d'énergie

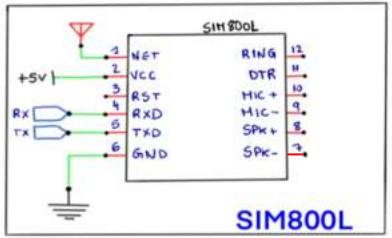


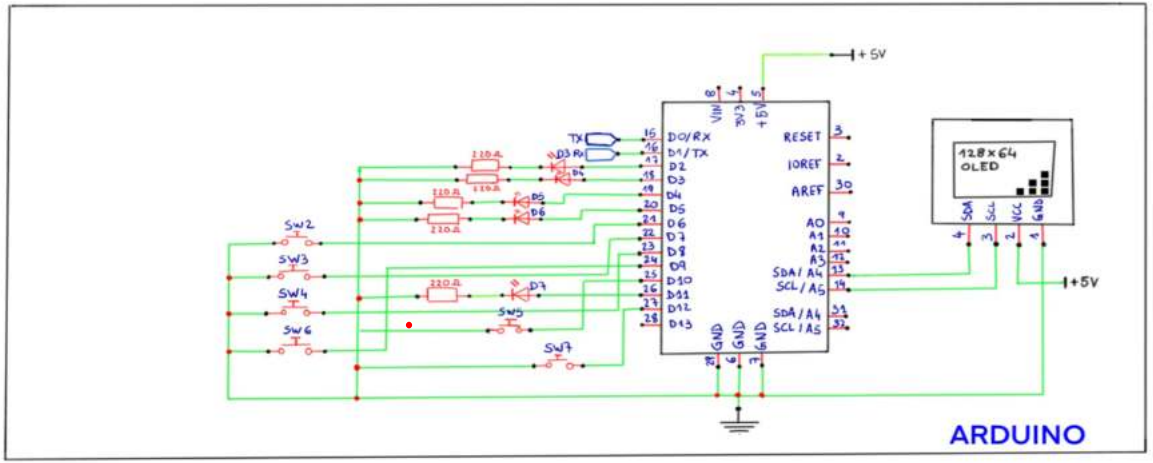
Batteries Lithium-ion 18650

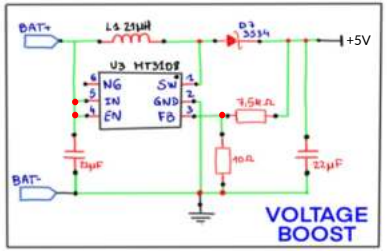
Haute densité énergétique, recharge rapide





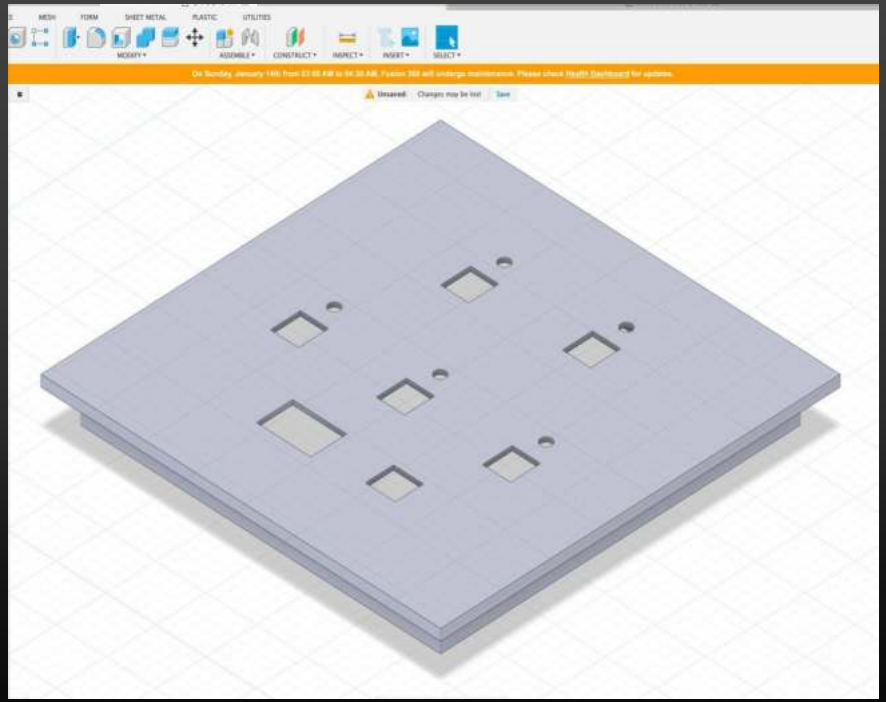




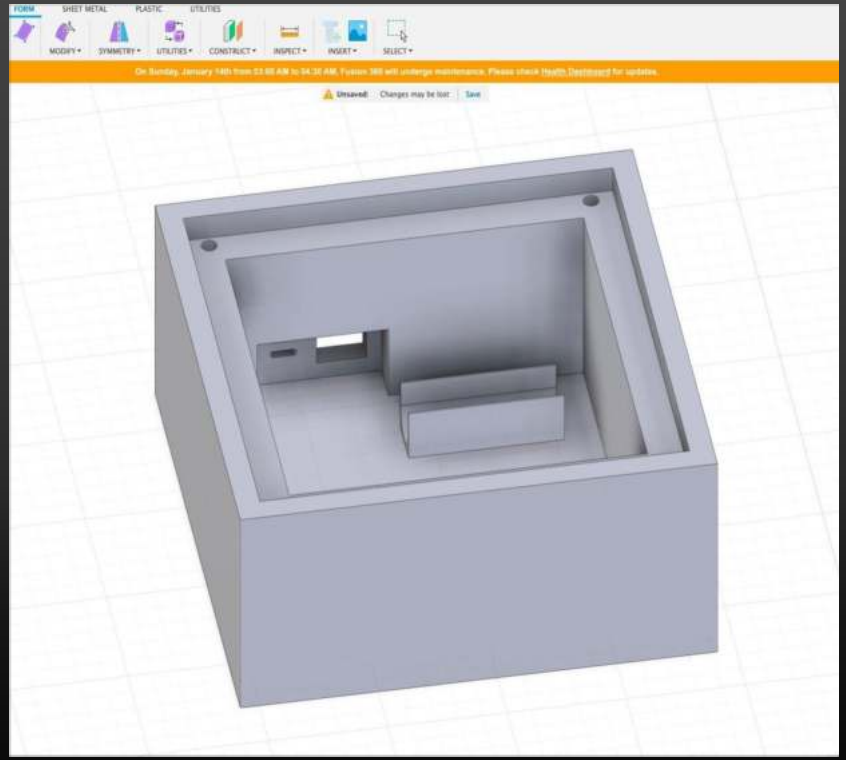


TITRE: Schématique TIPE	SCALE:
DATE: 2023-12-12	SHEET: 1/1
DRAWN BY: Durieux Lisa-Marie	

Conception boîtier



Rendu final du capuchon

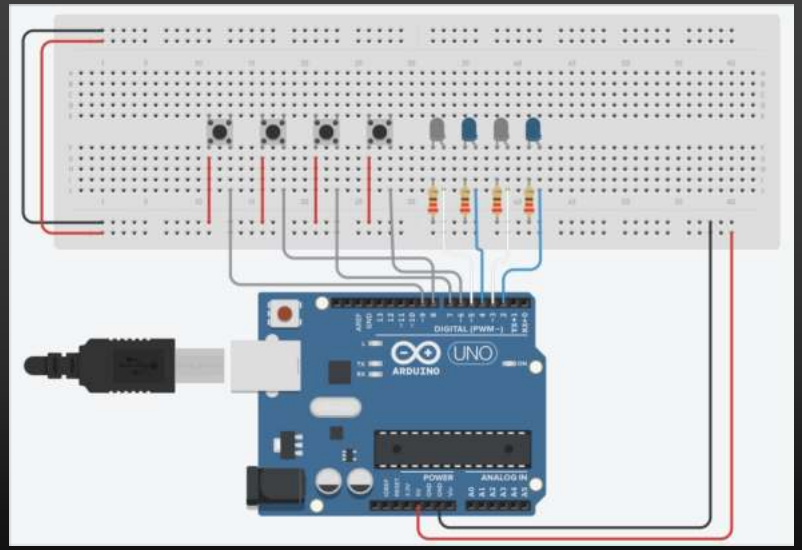


Rendu final du boitier

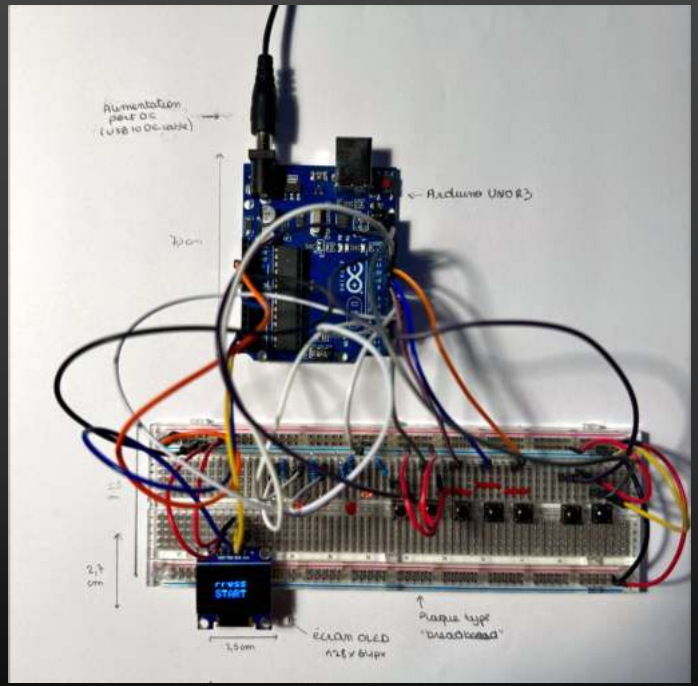
Démarche expérimentale

- Fabriquer l'appareil précédemment conçu
- Adapter le produit réel aux attentes du cahier des charges
- Tester afin de récolter des données

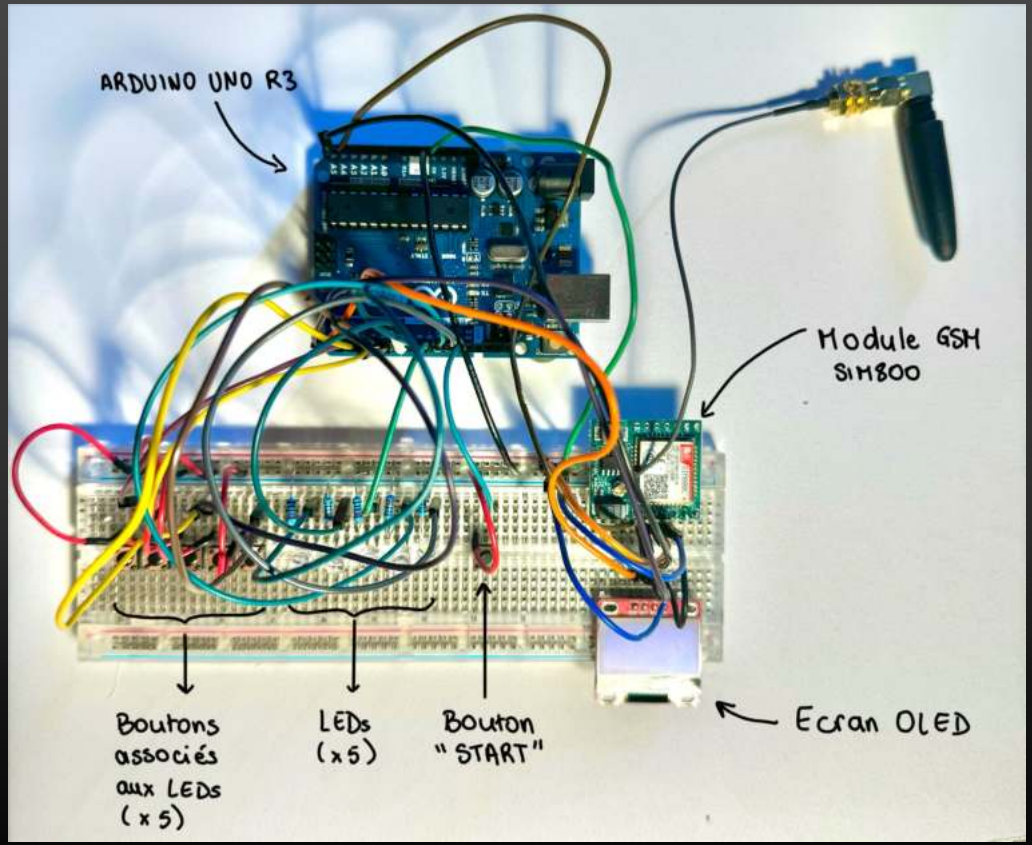
Premiers prototypes



Première CAO simplifiée sur une plaque de type ‘breadboard’

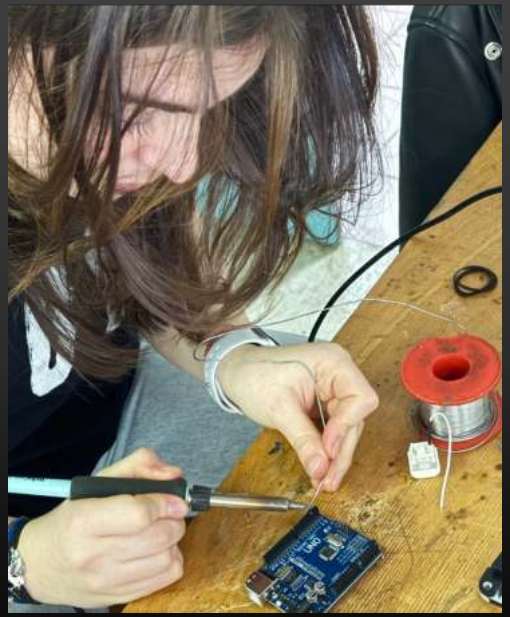


Premier prototype réel amélioré sur une plaque de type 'breadboard'



Prototype final sur une plaque de type 'breadboard'

Fabrication et montage de la maquette finale



Soudure

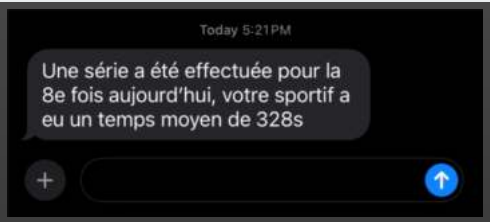


Ensemble soudé



Première recharge

Fonctionnement





Démarrage du dispositif



Mise en marche du dispositif, décompte + allumage aléatoire des LEDs



Fin de l'entraînement, l'écran affiche le temps de réponse du sujet

Le code de fonctionnement de l'ensemble LEDs/boutons/écran



Arduino Uno

sketch_may27c.ino

```
while (millis() - startTime < countdownDuration * 1000) { //loop that will go on until countdown is over
    if (remainingSeconds != (countdownDuration - (millis() - startTime) / 1000)){
      int remainingSeconds = (countdownDuration - (millis() - startTime) / 1000);
      display.clearDisplay();
      display.setTextSize(2);
      display.setTextColor(SSD1306_WHITE);
      display.setCursor(10, 20);
      display.print("Countdown: ");
      display.print(remainingSeconds);
      display.display();
    }

    if (currentLed == -1) {
      currentLed = random(numberOfLeds);
      while (currentLed == lastLed) {
        currentLed = random(numberOfLeds);
      } // Generate a new random LED index until it is different from the last one

      digitalWrite(ledPins[currentLed], HIGH);
      lastLed = currentLed; // Update the last lit LED
      startPressTime = millis();
    }

    if (digitalRead(buttonPins[currentLed]) == LOW) {
      digitalWrite(ledPins[currentLed], LOW);
      currentLed = -1;
      pressTimes[pressCount] = millis() - startPressTime;
      pressCount++;
      while(digitalRead(buttonPins[currentLed]) == LOW);
      //delay(1000); // Delay for 1 second after turning off the LED
    }
  }
  for (int i = 0; i < numberOfLeds; i++) {
    digitalWrite(ledPins[i], LOW);
  }
```

Output

```
Sketch uses 18098 by
Global variables use
```

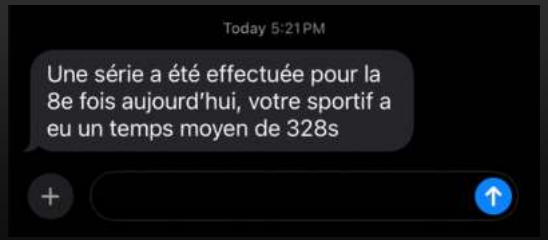
Le code de calcul du temps de réponse moyen :

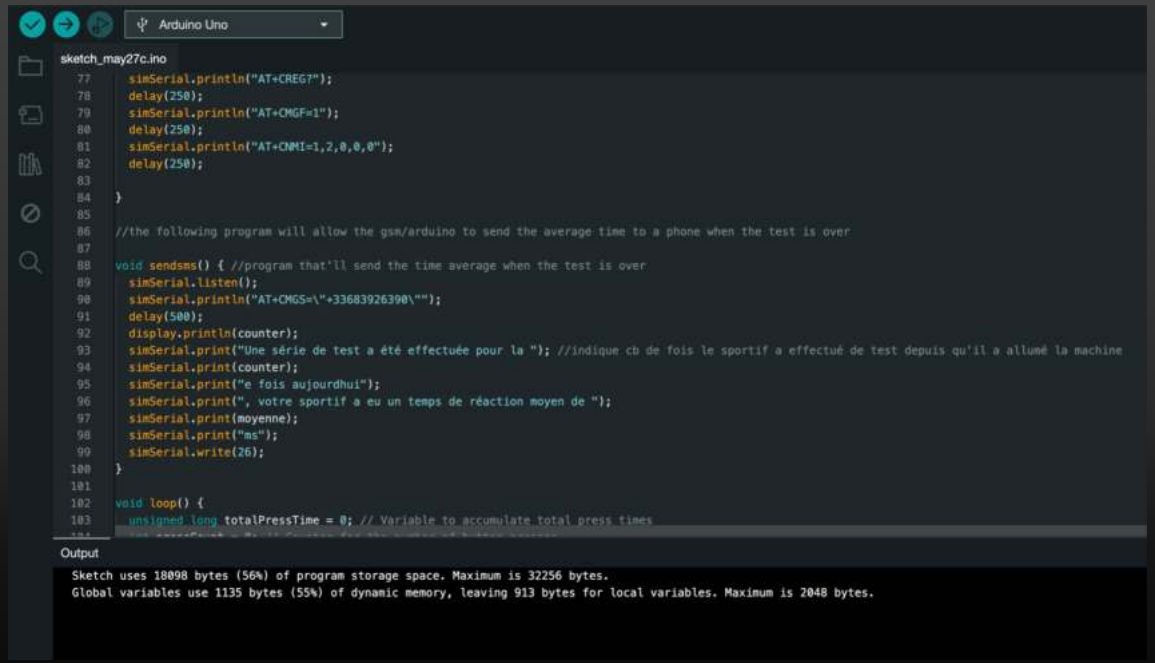
```
    moyenne = 0;
    for(int i = 0; i < pressCount; i++) {
      moyenne = moyenne + pressTimes[i];
    }
    moyenne = moyenne / pressCount;
    moyenneSec = moyenne * 0.001; //changes time average from milesonds to seconds

```

Le code de partage des données récupérées







Produit final



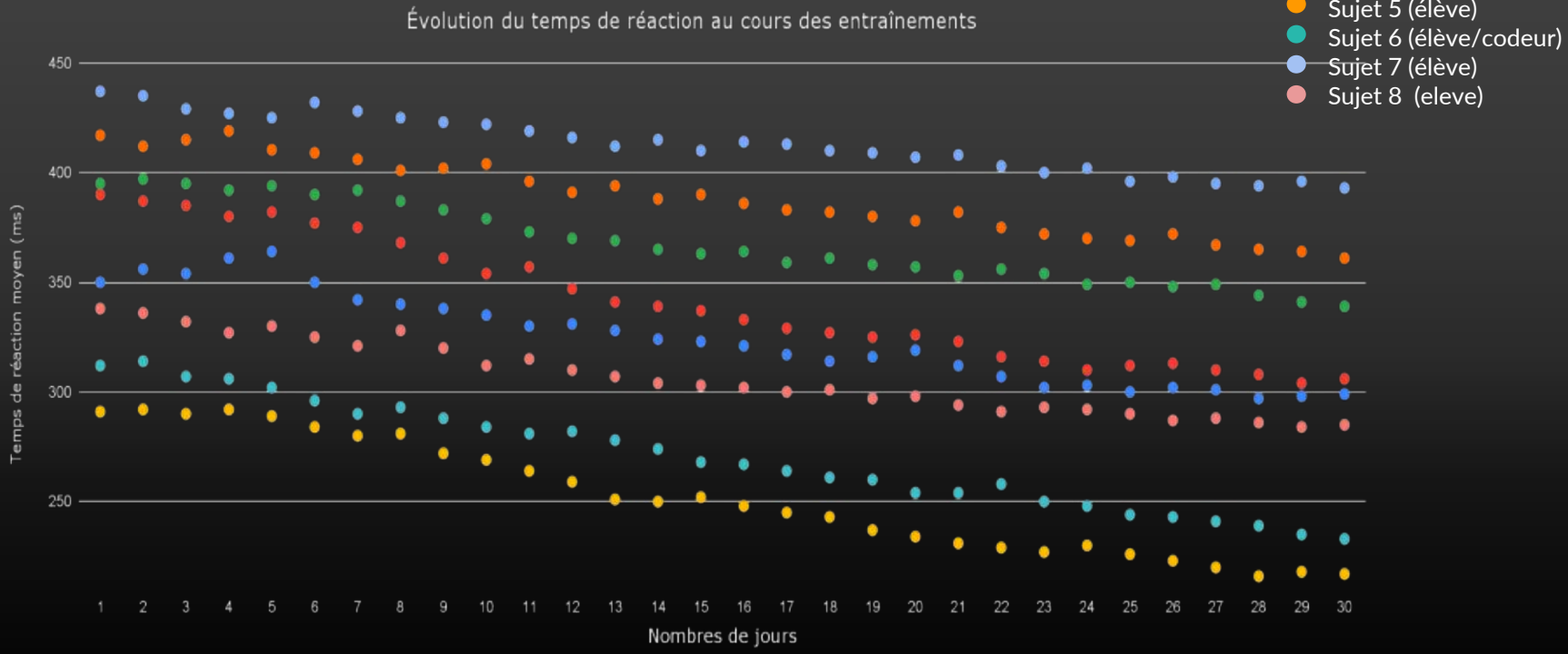


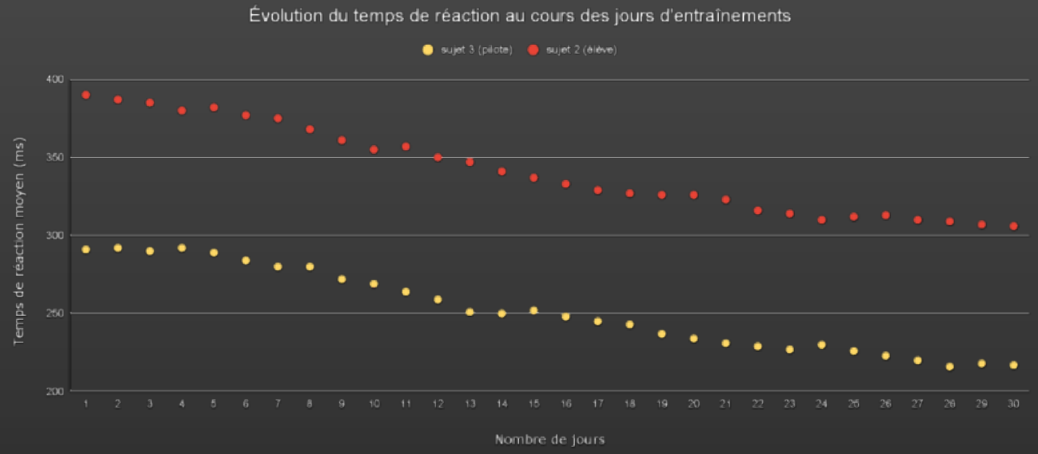
Coût total

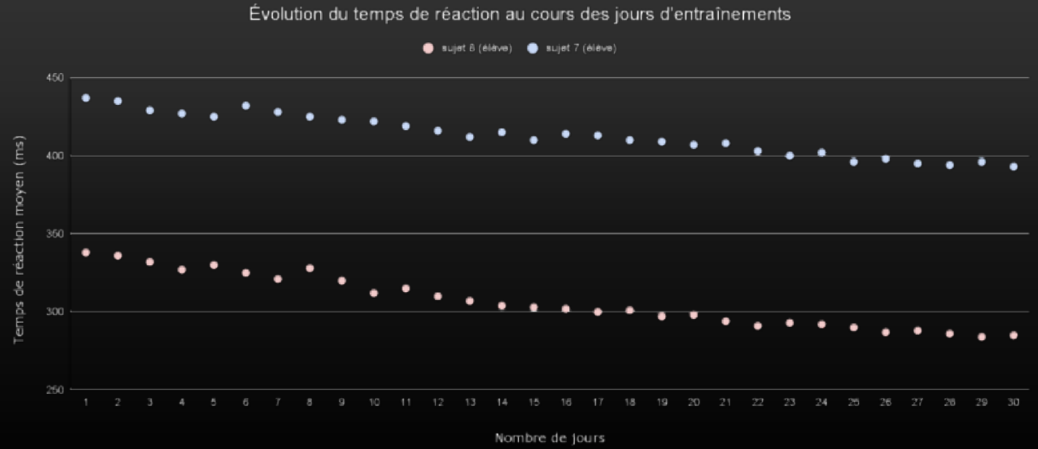
1) Carte Arduino : 14€
2) Batterie : 5€
3) GSM : 3€
4) Autres composants : 8€
5) Boitier : 15€

Total : 45€

Observables







Fortes évolutions

Sujet 2 (élève)
Temps de réaction (moyenne) : 341,2 ms
Écart type : 28,5
Évolution en 30 jours : 86 ms

Sujet 3 (pilote)
Temps de réaction (moyenne) : 252 ms
Écart type : 26,3
Évolution en 30 jours : 76 ms

Faibles évolutions

Sujet 7 (élève)
Temps de réaction (moyenne) : 413,1 ms
Écart type : 13,11
Évolution en 30 jours : 44 ms

Sujet 8 (élève)
Temps de réaction (moyenne) : 306,53 ms
Écart type : 16,85
Évolution en 30 jours : 54 ms

Conclusion : Réponse à la problématique

- Cahier des charges respecté
- Tests possibles
- Amélioration du temps de réaction au cours des entraînements
- Diminution du temps de réaction en un mois de 10% à 25%
- Diminution moyenne du temps de réaction en un mois de 65,5ms

Conclusion : Améliorations possibles

- Trouver un protocole de communication plus accessible, plus sûr sur le long terme et autant abordable que le réseau 2G utilisé (LoraWAN ou 4G, coût plus élevés)
- Améliorer le code de fonctionnement du dispositif (mise en place d'un système de "malus", indicateur de batterie faible, mise en place d'un système adaptable à chaque utilisateur...)
- Créer une interface mobile facile à prendre en main qui n'empêche pas de pouvoir utiliser le dispositif sans téléphone si souhaité

Annexes

Dimensions du boitier final: 17.8 x 18 x 11

Efficacité du convertisseur boost : 91%

Batterie : 11.1 Wh

Autonomie de charge : 1 jour

Durée de charge : 1 à 2h

```
//les lignes suivantes importent les librairies necessaires pour utiliser l'écran
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h> //for better communication between oled display and arduino
#include <SoftwareSerial.h> //for gsm module
#include <EEPROM.h>


//les lignes suivantes servent a définir la taille de l'écran (il est en 128x64px)
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1); //initializes the object "display" with the width and height we defined earlier

int ledPins[] = {2, 3, 4, 5, 12} ; // pins connected to the leds
int buttonPins[] = {6, 7, 8, 9, 11 }; // pins connected to buttons
int numberOfLeds = 5; //defines the number of leds used
int currentLed = -1; // indentation pour les leds : -1 = aucune led n'est allumée
int lastLed = -1; // keeps track of which led was last lit so no led is turned on twice in a row
const int startButtonPin = 10; // start button is connected to pin 10, when this button is pressed, it will start the program and make it run for 30 secs
bool startButtonPressed = false; // booléen pour nous aider a indiquer si le bouton start est appuyé ou non
bool countdownFinished = false; //booléen that will keep track of whether or not the countdown is off
int pressCount = 0; // to help keep track of the number
int countdownDuration = 30; //sets countdown time to 30 second
unsigned long startPressTime = 0;
int pressTimes[100]; //a list to store all the values of the average time.
unsigned long startTime = 0;
int remainingSeconds;
unsigned long moyenne = 0;
unsigned long moyenneSec = 0;

int counterAddress = 0;    // EEPROM address to store the counter value
int counter = 0;           // Counter variable

//next lines will be used for the gsm module
const byte SIMTX = 0 ;
const byte SIMRX = 1 ;
SoftwareSerial simSerial(SIMTX,SIMRX); //assigns sim gsm module to TX and RX pins



 void setup() {
   randomSeed(analogRead(0));

   for (int i = 0; i < numberOfLeds; i++) {
     pinMode(ledPins[i], OUTPUT);
     digitalWrite(ledPins[i], LOW); // all leds are off
     pinMode(buttonPins[i], INPUT_PULLUP);
   }

   pinMode(startButtonPin, INPUT_PULLUP); //to detect whether start button is pressed or not and activate its pull up resistor so that the button is powered well

  //following lines will make let us know if display's initialization was unsuccesful
   if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { //initializes oled display
     Serial.println(F("SSD1306 allocation failed")); //displays message if initaillization failed
     for (;;); //infinite loop that stops the program from running if display initialization fails
   }

   display.clearDisplay(); //clears display from anything that was written before
   display.setTextSize(2); //text size
   display.setTextColor(SSD1306_WHITE); //text color
   display.setCursor(30, 20); //text position on screen
   display.println("Press");
   display.setCursor(30,40);
   display.println("START");
   display.display(); //displays "Press START:

   //next line will initialize the gsm module
   simSerial.listen();
   delay(1000);
   simSerial.println("AT");
   delay(250);
   simSerial.println("AT+CSQ");
   delay(250);
   simSerial.println("AT+CCID");
   delay(250);
   simSerial.println("AT+CREG?");
   delay(250);
   simSerial.println("AT+CMGF=1");
   delay(250);
   simSerial.println("AT+CNMI=1,2,0,0,0");
   delay(250);

}
//the following program will allow the gsm/arduino to send the average time to a phone when the test is over

void sendsms() { //program that'll send the time average when the test is over
   simSerial.listen();
   simSerial.println("AT+CMGS=\"+33683926398\"");
   delay(500);
   display.println(counter);
   simSerial.print("Une série de test a été effectuée pour la "); //indique cb de fois le sportif a effectué de test depuis qu'il a al
   simSerial.print(counter);
   simSerial.print("e fois aujourdhui");
   simSerial.print(", votre sportif a eu un temps de réaction moyen de ");
   simSerial.print(moyenne);
   simSerial.print("ms");
   simSerial.write(26);
}

void loop() {
   unsigned long totalPressTime = 0; // Variable to accumulate total press times
   int pressCount = 0; // Counter for the number of button presses

   if (digitalRead(startButtonPin) == LOW ) { //checks if the button is pressed and if it has not been pressed previously ; code below
     counter++; // counter+=1
     Serial.print("Counter value");
     Serial.print(counter);
     EEPROM.write(counterAddress, counter);

     startTime = millis(); //record time since arduino started running
     pressCount = 0;
     remainingSeconds = countdownDuration;
     currentLed = -1;
     lastLed = -1;

     display.clearDisplay();
     display.setTextSize(2);
     display.setTextColor(SSD1306_WHITE);
     display.setCursor(10, 20);
     display.print("Countdown: ");
     display.print(remainingSeconds);
     display.display();

     unsigned long startTime = millis(); //record time since arduino started running
     int countdownDuration = 30; // sets countdown time to 30 seconds

   while (millis() - startTime < countdownDuration * 1000) { //loop that will go on until countdown is over
       if (remainingSeconds != (countdownDuration - (millis() - startTime) / 1000)){
         int remainingSeconds = (countdownDuration - (millis() - startTime) / 1000);
         display.clearDisplay();
         display.setTextSize(2);
         display.setTextColor(SSD1306_WHITE);
         display.setCursor(10, 20);
         display.print("Countdown: ");
         display.print(remainingSeconds);
         display.display();
       }
```

```
    if (currentLed == -1) {
      currentLed = random(numberOfLeds);
      while (currentLed == lastLed) {
        currentLed = random(numberOfLeds);
      }  // Generate a new random LED index until it is different from the last one

      digitalWrite(ledPins[currentLed], HIGH);
      lastLed = currentLed; // Update the last lit LED
      startPressTime = millis();
    }

    if (digitalRead(buttonPins[currentLed]) == LOW) {
      digitalWrite(ledPins[currentLed], LOW);
      currentLed = -1;
      pressTimes[pressCount] = millis() - startPressTime;
      pressCount++;
      while(digitalRead(buttonPins[currentLed]) == LOW);
      //delay(1000); // Delay for 1 second after turning off the LED
    }
  }
  for (int i = 0; i < numberOfLeds; i++) {
    digitalWrite(ledPins[i], LOW);
  }
  Serial.println(pressCount);
  for(int i = 0; i < pressCount; i++) {
    Serial.print(pressTimes[i]);
    Serial.print(";");
  }
  Serial.println();
  moyenne = 0;
  for(int i = 0; i < pressCount; i++) {
    moyenne = moyenne + pressTimes[i];
  }
  moyenne = moyenne / pressCount;
  moyenneSec = moyenne * 0.001; //changes time average from milesonds to seconds

    // Countdown finished, set the flag and turn off LEDs
    if (millis() - startTime >= countdownDuration * 1000 && !countdownFinished) {
      countdownFinished = true;
      for (int i = 0; i < numberOfLeds; i++) {
        digitalWrite(ledPins[i], LOW);
      }
      display.clearDisplay();
      display.setTextSize(2);
      display.setTextColor(SSD1306_WHITE);
      display.setCursor(30, 20);
      display.println("Finish");
      display.display();
      delay(2000); // Delay for 5  second to display
      unsigned long averagePressTime = totalPressTime / pressCount;
      display.clearDisplay();
      display.setTextSize(1);
      display.setTextColor(SSD1306_WHITE);
      display.setCursor(30, 30);
      display.println("Score : ");
      display.setTextSize(2);
      display.setTextColor(SSD1306_WHITE);
      display.setCursor(30, 40);
      display.print(moyenne);
      display.print("ms");
      display.display();
      delay(5000); // Display the average press time for 5 seconds before showing "Finish" and "Press" message again
      Serial.println(moyenne);
      display.display();
      delay(2000); // Display the average press time for 2 seconds before showing "Finish" and "Pres
      totalPressTime = 0; // Reset total press time
      pressCount = 0; // Reset press count
      display.clearDisplay();
      sendsms(); //will send the avg time
    }
  }

  // Display "Press start" until start button is pressed again
  while (digitalRead(startButtonPin) == HIGH) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(30, 20);
    display.println("Press");
    display.setCursor(30,40);
    display.println("START");
    display.display();
    delay(100); // Add a short delay to avoid flickering
  }

  // Reset variables before starting the countdown again
  currentLed = -1;
  lastLed = -1;
  startButtonPressed = false;
  countdownFinished = false;

}
```